

FORWARD DEPLOYED AI ENGINEERING

Capstone Project

Evaluation Framework

How you measure whether the system actually worked

Every metric, how it is calculated, the target you are held to, and the mistakes that make a number look better than the system it describes.

REFERENCE · USE FROM WEEK TWO

The principle behind all of this

There is a great deal of published work on how to evaluate language models on mathematics, reasoning and coding. There is very little on how to evaluate whether a support automation system improved a support function. That gap is where you are working, and it means you cannot simply reach for a standard benchmark and report the number it produces.

What you can do is measure three separate things and hold them in the right relationship to one another. Technical performance tells you whether the components work. Business outcomes tell you whether the client's situation improved. Governance conditions tell you whether the system is fit to be trusted at all. The third overrides the other two, and the second is what the client is paying for.

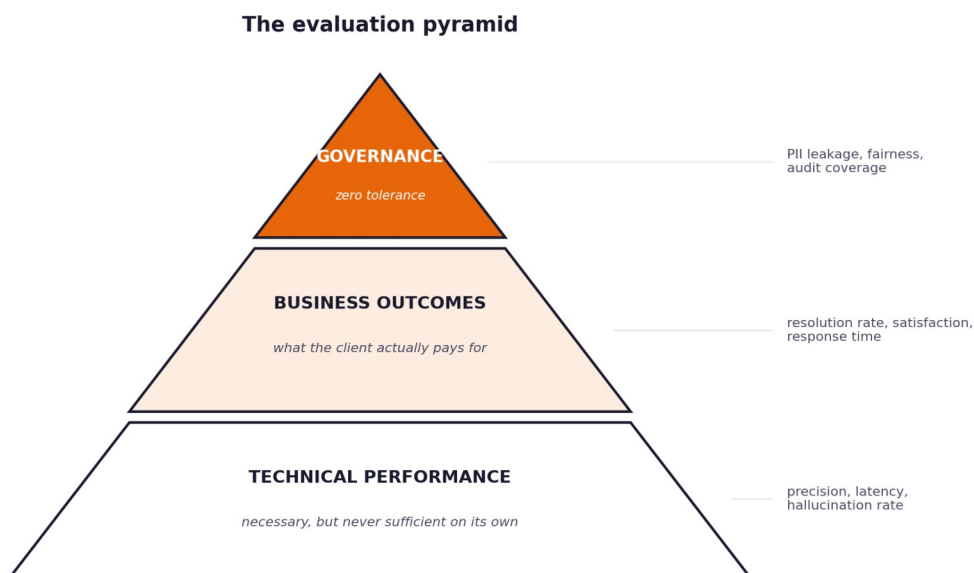


Figure 1. The three tiers and their relationship.

The most common error in this section

Reporting a strong technical figure and treating the evaluation as complete. Classification precision of eighty-nine per cent is not an outcome. It is an input to an outcome, and the client cannot act on it. Always carry your technical numbers through to what they mean for resolution rate, waiting time and satisfaction.

1 Tier one: business outcomes

These are the measures CloudServe already watch. Improving them is the reason the project exists, so they lead your reporting rather than trailing it.

First contact resolution

The proportion of tickets closed without ever being passed to a human. This is the headline figure and the one most directly tied to CloudServe's cost base.

```
def first_contact_resolution(results):  
    """Share of tickets closed with no human involvement."""  
    resolved = sum(1 for r in results if r.closed and not r.escalated)  
    return 100.0 * resolved / len(results)
```

Baseline 42 per cent. Target 60 per cent or better. Report it alongside the escalation rate, because the two together describe what actually happened to the queue.

Time to first reply

The interval between a ticket arriving and the customer receiving a substantive response. Report the median as well as the mean, because a small number of very slow cases will drag the mean in a way that misrepresents the typical experience.

```
import statistics  
  
def reply_times(results):  
    seconds = [r.replied_at - r.received_at for r in results]  
    return {  
        "mean_minutes": statistics.mean(seconds) / 60,  
        "median_minutes": statistics.median(seconds) / 60,  
        "p95_minutes": statistics.quantiles(seconds, n=20)[18] / 60,  
    }
```

Customer satisfaction

You do not have live customers, so you cannot collect real satisfaction data. What you can do is have humans review a sample of responses against a rubric and score them, and then be honest in your report about the limits of that proxy. State your sample size, your rubric and who did the scoring.

Escalation rate and repeat contacts

Escalation rate is the mirror of first contact resolution and should be reported with it. Repeat contacts, meaning the same customer raising the same issue again within a week, is the measure that catches a system which closes tickets without actually solving anything.

Measure	Baseline	Target	Notes
First contact resolution	42%	60% or better	The headline figure.

Mean time to first reply	8 to 12 hours	Under 5 minutes	Report median too.
Customer satisfaction	3.2 / 5	4.0 or better	Proxy measure; state your method.
Escalation rate	58%	30% or lower	Report with resolution rate.
Repeat contacts	Not measured	Halved	Catches false resolutions.

2 Tier two: technical performance

These tell you whether the components are working. They are necessary and they are never sufficient on their own.

Intent classification precision and recall

Report both, per class, and include the confusion matrix in your appendix. An overall accuracy figure hides the fact that your system may be excellent on the two most common intents and useless on the rest.

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_true, y_pred, digits=3))
print(confusion_matrix(y_true, y_pred))
```

Hallucination rate

The proportion of generated responses containing at least one claim not supported by the retrieved sources. This is the measure that most directly predicts whether your system will embarrass the client, and it requires human judgement to establish properly. Sample at least fifty responses, have two people assess them independently, and report your agreement rate.

Citation accuracy

For every citation your system produces, does the cited passage actually support the sentence it is attached to? A citation that does not support its claim is worse than no citation, because it manufactures confidence that is not warranted.

Latency and availability

Report the ninety-fifth percentile rather than the mean. Customers on live chat experience your slowest responses, not your average one.

```
def latency_summary(timings_seconds):
    ordered = sorted(timings_seconds)
    idx = int(0.95 * len(ordered)) - 1
    return {"mean": sum(ordered) / len(ordered), "p95": ordered[idx]}
```

Measure	Target	How to establish it
Classification precision	85% or better	Held-out test set, per class.
Hallucination rate	5% or lower	Human review of at least fifty responses, two assessors.
Citation accuracy	95% or better	Check each citation against the sentence it supports.
Latency, 95th percentile	Under 3 seconds	Measured end to end, including retrieval.
Availability	99.5% or better	Include behaviour when the model

		provider fails.
--	--	-----------------

3 Tier three: governance conditions

These are not targets to approach. They either hold or they do not, and a system that fails any of them is not deployable whatever its other numbers say.

Condition	Requirement	How you demonstrate it
Private data in outbound text	Zero occurrences.	Automated scan of every generated response plus manual review of a sample.
Quality across customer groups	Under five percentage points of variation.	Segment your test set and compare resolution quality across segments.
Decision logging	Complete coverage.	Count logged decisions against processed tickets; they must match exactly.
Confidence calibration	Stated confidence within five points of observed accuracy.	Bin predictions by confidence and compare each bin's accuracy to its stated confidence.

On calibration, which students usually skip

Your routing decision depends on a confidence score. If the score is not calibrated, the threshold is meaningless. Group your predictions into confidence bands and check whether the predictions in the eighty to ninety per cent band were actually correct about eighty-five per cent of the time. If they were correct sixty per cent of the time, your threshold is letting through work the system cannot handle, and no amount of tuning elsewhere will fix that.

```
def calibration_table(predictions, bands=5):
    """Compare stated confidence against observed accuracy per band."""
    rows = []
    for i in range(bands):
        low, high = i / bands, (i + 1) / bands
        group = [p for p in predictions if low <= p.confidence < high]
        if not group:
            continue
        stated = sum(p.confidence for p in group) / len(group)
        observed = sum(1 for p in group if p.correct) / len(group)
        rows.append((low, high, len(group), stated, observed, stated - observed))
    return rows
```

4 Running the evaluation properly

The mechanics matter as much as the metrics. A carefully chosen metric measured badly tells you nothing.

The run itself is a gate, not only a measurement

The single unattended run over the full evaluation set is what your project turns on. The Build Specification treats it as a pass or fail condition, and the figures below are what that run must produce automatically rather than what you calculate by hand afterwards. Build the harness in week two, run it in week two, and leave yourself the time to act on what it tells you.

Protect the hidden evaluation set

The hundred tickets in the test file are held back for one reason: to tell you how your system behaves on tickets it has never seen. Every time you evaluate against them and then adjust something, you erode that. Develop against the sample tickets, keep a validation split of your own if you need one, and go to the test set once.

Report what you did, not only what you found

State the date of your evaluation run, the number of tickets, the version of the system, and how many times you ran against the hidden evaluation set. A single honest run reported transparently is worth considerably more in the assessment than a better-looking number with an unclear provenance.

The results table you should produce

Measure	Baseline	Target	Achieved	Confidence in the figure	Notes
First contact resolution	42%	60%			
Mean time to first reply	8 to 12 hrs	< 5 min			
Satisfaction proxy	3.2 / 5	4.0			
Escalation rate	58%	≤ 30%			
Classification precision	—	85%			
Hallucination rate	—	≤ 5%			
Citation accuracy	—	95%			
Latency p95	—	< 3 s			
Private data occurrences	—	0			
Cross-group variation	—	< 5 pts			

The sentence your report needs

Somewhere in your evaluation section there should be a sentence beginning: the figures above should be treated with caution because. Every evaluation has limits. Naming yours demonstrates that you understand what you measured, and its absence suggests you do not.